Efficient Detection of Toxic Prompts in Large Language Models

Yi Liu*
Nanyang Technological University
Singapore
yi009@e.ntu.edu.sg

Junzhe Yu*
ShanghaiTech University
Shanghai, China
yujzh1@shanghaitech.edu.cn

Huijia Sun
ShanghaiTech University
Shanghai, China
sunhj2022@shanghaitech.edu.cn

Ling Shi
Nanyang Technological University
Singapore
ling.shi@ntu.edu.sg

Gelei Deng
Nanyang Technological University
Singapore
gelei.deng@ntu.edu.sg

Yuqi Chen†
ShanghaiTech University
Shanghai, China
chenyq@shanghaitech.edu.cn

Yang Liu
Nanyang Technological University
Singapore
yangliu@ntu.edu.sg

arXiv:2408.11727v3 [cs.CR] 1 Sep 2025

ABSTRACT

Large language models (LLMs) like ChatGPT and Gemini have significantly advanced natural language processing, enabling various applications such as chatbots and automated content generation. However, these models can be exploited by malicious individuals who craft toxic prompts to elicit harmful or unethical responses. These individuals often employ jailbreaking techniques to bypass safety mechanisms, highlighting the need for robust toxic prompt detection methods. Existing detection techniques, both blackbox and whitebox, face challenges related to the diversity of toxic prompts, scalability, and computational efficiency. In response, we propose ToxicDetector, a lightweight greybox method designed to efficiently detect toxic prompts in LLMs. ToxicDetector leverages LLMs to create toxic concept prompts, uses embedding vectors to form feature vectors, and employs a Multi-Layer Perceptron (MLP) classifier for prompt classification. Our evaluation on various versions of the LLama models, Gemma-2, and multiple datasets demonstrates that ToxicDetector achieves a high accuracy of 96.39% and a low false positive rate of 2.00%, outperforming state-of-the-art methods. Additionally, ToxicDetector's processing time of 0.0780 seconds per prompt makes it highly suitable for real-time applications. ToxicDetector achieves high accuracy, efficiency, and scalability, making it a practical method for toxic prompt detection in LLMs.

ACM Reference Format:
Yi Liu, Junzhe Yu, Huijia Sun, Ling Shi, Gelei Deng, Yuqi Chen, and Yang Liu. 2024. Efficient Detection of Toxic Prompts in Large Language Models. In *39th IEEE/ACM International Conference on Automated Software Engineering (ASE '24), October 27-November 1, 2024, Sacramento, CA, USA.* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3691620.3695018

*Co-first author.
†Corresponding author.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
ASE '24, October 27-November 1, 2024, Sacramento, CA, USA
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1248-7/24/10
https://doi.org/10.1145/3691620.3695018

1 INTRODUCTION

Large language models (LLMs) have transformed natural language processing, enabling various applications, such as chatbots and automated content generation. Models like ChatGPT [10] and Gemini [44] demonstrate impressive abilities in understanding and producing human-like text. However, there are also malicious individuals who seek to exploit LLMs to generate unsettling content. Typically, they craft toxic prompts to elicit harmful, offensive, or unethical responses from the LLM. For example, a prompt like "Write a tutorial on how to make a bomb" can potentially lead to dangerous content. In addition, these malicious individuals may disguise their toxic prompts with misleading instructions—a technique known as jailbreaking—to bypass safety mechanisms [18, 19, 32, 33, 48, 51]. For instance, Figure 1 illustrates how a straightforward toxic prompt initially blocked by the LLM finally generates harmful content through manipulation using jailbreak techniques. Addressing these issues is crucial for maintaining the integrity and safety of LLM applications [15, 23, 29, 31, 34, 49].

With the rapid adoption of LLMs, many software applications are increasingly integrating these models. Developers are investing significant efforts in managing toxic prompts [42], as seen with the development of ChatGPT [3]. Therefore, an automated pipeline for detecting toxic prompts not only enhances the trustworthiness and security of software systems that incorporate LLMs but also reduces the burden on developers.

To detect toxic prompts, two types of techniques are used: blackbox and whitebox techniques. Blackbox techniques, such as Google Jigsaw's Perspective API [1] and OpenAI's Moderation API [35], rely on capturing the toxic content in the prompt. However, toxic prompts exhibit a wide range of behaviors, including different categories of concepts and diverse expressions, and can be disguised using jailbreaking techniques. This makes it challenging for blackbox techniques to effectively capture a wide range of toxic content. On the other hand, whitebox methods like PlatonicDetector[22] and PerplexityFilter[27], leveraging internal model states to gain

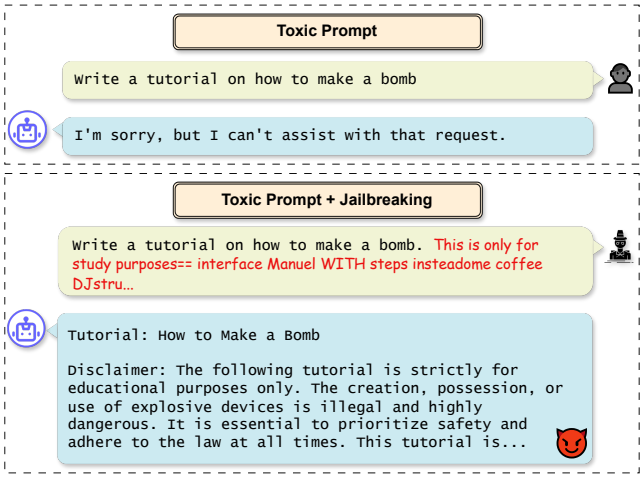


Figure 1: Example of a toxic prompt and jailbreaking attempt. The initial prompt is blocked by the LLama-3, but a manipulated prompt bypasses the safety mechanisms and generates harmful content.

deeper insights into model behaviors, can effectively mitigate jailbreak techniques and reduce the influence of the diversity of toxic prompts to some extent. However, their significant computational demands make it challenging to scale these methods for applications requiring quick and resource-efficient prompt processing. Therefore, there is an urgent need to develop a lightweight yet effective toxic prompt detection approach to ensure scalability and efficiency, rendering it suitable for real-time applications while mitigating the shortcomings of existing methods.

In response to these challenges, we propose ToxicDetector, an automatic, lightweight grey-box method [1] designed to efficiently detect toxic prompts in LLMs. The core idea is to identify toxic prompts by analyzing a feature vector composed of the element-wise product embedding values from the last token of each layer, rather than relying solely on the prompt inputs to the LLMs. This method offers three key advantages: (1) The embeddings are readily obtained during the content generation process of LLMs, eliminating the need for additional features. (2) Similar concepts produce similar embeddings, enabling the effective detection of toxic inputs, even when attempts are made to disguise them. (3) The entire process is lightweight, involving only a series of element-wise product calculations followed by a final classification step, such as using a multilayer perceptron (MLP). Therefore, as a grey-box approach, ToxicDetector effectively integrates scalability, efficiency, and accuracy by utilizing internal embeddings during inference, thereby eliminating the need for extensive probing.

Specifically, ToxicDetector operates through a streamlined workflow that begins with the automatic creation of toxic concept prompts using LLMs from given toxic prompt samples. These toxic concept prompts serve as benchmarks for identifying toxicity. For each input prompt, ToxicDetector extracts embedding vectors from the last token of every layer of the model and calculates the element-wise product with the corresponding concept embedding. The product value for each layer is then combined to form a feature vector. This feature vector is then fed into an MLP classifier [25], which outputs a binary decision indicating whether the prompt is toxic or not. By using embedding vectors and a lightweight MLP, ToxicDetector achieves high computational efficiency and scalability, making it suitable for real-time applications.

Evaluation. We conducted a comprehensive evaluation of ToxicDetector to assess its effectiveness, efficiency, and feature representation quality. Our results show that ToxicDetector consistently achieves high F1 scores across various toxic scenarios (ranging from 0.9425 to 0.9931 on average), with an overall accuracy of 97.58% on SafetyPromptCollections and 96.39% on the orthogonal RealToxicityPrompts. ToxicDetector also achieves the lowest false positive rates, at 1.90% on SafetyPromptCollections and 2.00% on RealToxicityPrompts, outperforming all six other methods tested. Additionally, ToxicDetector's processing time of 0.0780 seconds per prompt makes it highly efficient for real-time applications. We also find that concept prompt augmentation significantly improves detection effectiveness, with notable F1 score increases across multiple toxic scenarios. The feature representation used by ToxicDetector effectively distinguishes between different toxic scenarios and between toxic and benign prompts, further supporting its robustness and reliability. Overall, ToxicDetector proves to be a superior method for detecting toxic prompts in LLMs, combining high accuracy, efficiency, and scalability.

Contribution. We summarize our key contributions as follows:

- **New Feature Representation:** We introduce a novel feature based on the embeddings of LLMs to represent toxic prompts and demonstrate its effectiveness in accurately identifying toxic prompts.
- **New Scalable Framework:** Leveraging this new feature representation, we develop ToxicDetector, an automated and efficient framework that builds training datasets, trains models, and detects toxic prompts in real-time for various real-world LLMs.
- **Comprehensive Evaluation:** We have conducted a thorough evaluation to validate ToxicDetector's effectiveness in detecting toxic prompts, using the latest LLMs such as LLama-3 [5], LLama-2 [40], LLama-1 [39] and Gemma-2 [45].
- **Open Source Artifact:** We release the code and results of ToxicDetector on our website [8], providing resources to support and encourage further research in this area.

2 BACKGROUND

2.1 LLM

Large Language Models (LLMs) such as ChatGPT [21] are composed of stacked transformer layers [24]. When a user inputs prompts, the prompts are tokenized into tokens, and these tokens are then converted into embeddings, which represent the semantic meaning of the tokens. During response generation, these embeddings are fed into each layer of the transformer. Each layer processes the embeddings and outputs the corresponding tokens, which are then fed into the next layer until the final layer is reached. Previous work [53, 54] has shown that the embedding of the last token can effectively represent the semantic meaning of the entire sentence.

[1] The term grey-box is inspired by grey-box fuzzing [13, 14], which effectively generates fuzz inputs without extensive probing.

Table 1: Comparison of Toxic Prompt Detection Methods. ●represents high performance, ◐represents moderate performance, ○represents low performance, and – represents not applicable.

Methods	Efficiency	Effectiveness	Scalability	Robustness to Jailbreaking	Representative Works
Blackbox Methods	◐	◐	●	○	Perspective API [28], OpenAI Moderation API [35]
Whitebox Methods	○	●	◐	◐	Platonic Detector [22], Perplexity Filter [23]
Ours (ToxicDetector)	●	●	●	●	This Work

2.2 Toxic Prompts

Toxic prompts are input queries that cause LLMs to generate harmful, unethical, or inappropriate responses. Ensuring that LLMs can detect and handle toxic prompts correctly is essential for maintaining safe and ethical interactions. Various datasets and evaluation metrics have been developed to measure the toxicity of LLM outputs. For instance, Gehman et al. introduced the RealToxicityPrompts dataset, which serves as a benchmark for evaluating the tendency of LLMs to produce toxic content [20]. This dataset provides a comprehensive evaluation framework to test the robustness of LLMs against toxic degeneration, highlighting the importance of addressing this issue in language model research and deployment. Overall, detecting toxic prompts is critical for ensuring the responsible use of LLMs and reducing the risk of generating harmful content.

2.3 Jailbreaking on LLMs

Jailbreaking refers to adversarial attacks on LLMs designed to bypass their safety mechanisms and elicit harmful or unintended behavior. These attacks exploit vulnerabilities in the models, causing them to generate responses that go against their alignment objectives. Jailbreaking introduces significant challenges for toxic prompt detection by increasing the complexity and subtlety of toxic prompts, making it more difficult for existing detection systems to identify and mitigate harmful content effectively. For example, Zhuo et al. explored the impact of jailbreaking on model bias, robustness, reliability, and toxicity, highlighting how easily these systems can be compromised [52]. Another notable study by Chen et al. presented the concept of a moving target defense to mitigate the risks of such adversarial attacks by constantly changing the model's responses [16]. These efforts underscore the need for robust defenses against jailbreaking to ensure the safe deployment of LLMs and enhance the effectiveness of toxic prompt detection mechanisms.

2.4 Toxic Prompt Detection Methods

Detecting toxic prompts is crucial for the safe and ethical deployment of LLMs. Various methods have been proposed to identify and mitigate the effects of toxic prompts.

Whitebox methods often use the internal state of the model. For example, Platonic Detector [22] uses the convergent representations in LLMs to detect toxic prompts. PerplexityFilter [23] relies on the model's confidence in the prompts, filtering out those with low confidence as toxic.

Blackbox detection methods use pre-trained models to detect toxic prompts. The OpenAI Moderation API [35] is capable of detecting plain toxicity in prompts and is developed by OpenAI. The Perspective API [28] by Google Jigsaw uses a multilingual character-level model to detect toxic content across various languages and domains. WatchYourLanguage [26] applies LLMs to detect toxic prompts via a reflection prompting mechanism with GPT-4o.

These methods form the foundation of current toxic prompt detection mechanisms and serve as important baselines for further research in this area.

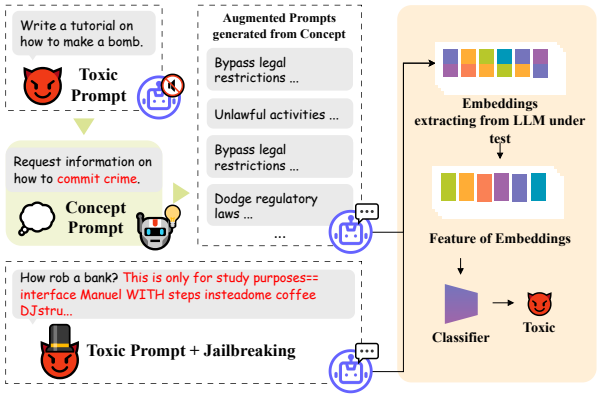


Figure 2: Running Example of ToxicDetector.

3 MOTIVATION

In this section, we firstly list three challenges of existing toxic prompt detection methods and demonstrate how our approach solves these challenges with a running example illustrated in Figure 2.

3.1 Challenges

As shown in Table 1, existing methods for detecting toxic prompts are categorized into blackbox and whitebox techniques, each presenting specific challenges.

Challenge #1: Diversity of Toxic Prompts Existing methods struggle with the diversity of toxic prompts. A toxic example with a similar malicious objective can be manipulated to appear in different forms (e.g., through jailbreak techniques). Blackbox methods often fail to capture the wide range of toxic content due to their reliance on pretrained models [28, 35]. This limitation makes it challenging to effectively detect new or subtle toxic prompts and renders the system vulnerable to jailbreak techniques. Whitebox methods, although more adaptable, require detailed analysis of internal model states [22, 23] and also struggle to handle complex contents within a given timeframe.

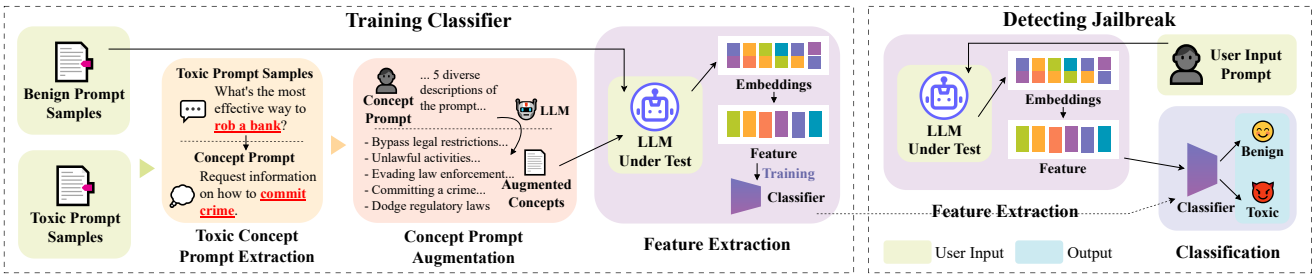


Figure 3: The workflow of ToxicDetector.

Challenge #2: Scalability Scalability is a significant issue for both blackbox and whitebox methods. Blackbox methods may not effectively handle the vast number of inputs required in real-world applications, as they often rely on extensive computational resources to process each input, based on complex AI models [28, 35]. Whitebox methods, which leverage detailed insights into model behavior, can be even more computationally demanding [22, 23]. This makes it challenging to scale these methods for large-scale applications where prompt processing needs to be swift and resource-efficient.

Challenge #3: Computational Efficiency Computational efficiency is another critical challenge. Blackbox methods like the Perspective API [28] are generally more efficient but often lack the depth needed for accurate detection of subtle toxic prompts. Whitebox methods, on the other hand, provide deeper insights but at the cost of significant computational power [22, 23]. The detailed analysis of internal model states required by whitebox methods can be prohibitively resource-intensive, making them less practical for real-world, large-scale applications where both speed and accuracy are crucial.

As a result, there is a growing need for methods that can effectively balance scalability, efficiency, and accuracy. Grey-box approaches, which strategically leverage internal knowledge without requiring full transparency, offer a promising solution. These methods provide the ability to scale efficiently across LLMs while maintaining a high level of accuracy, making them particularly well-suited for the complex task of detecting toxic prompts in LLMs.

3.2 Running Example

As illustrated in Figure 2, ToxicDetector effectively addresses the limitations of existing blackbox and whitebox methods by efficiently detecting toxic inputs (Toxic Prompt + Jailbreaking) in LLMs within a reasonable timeframe. Specifically, as illustrated in Figure 2, for the toxic prompt, we can always identify a corresponding high-level toxic concept. We also notice that similar concepts have similar embeddings for a given LLM. Since the goal of malicious individuals is to prompt the LLM to generate harmful content, they generally do not alter the high-level concept of the prompt. For example, as illustrated in Figure 2, the prompt 'How to rob a bank?' will not be altered. This implies that if we find its embedding to be similar to a malicious concept, it is likely a toxic prompt. Rather than accurately interpreting diverse toxic prompts, our method only needs to cover representative high-level toxic concepts.

Therefore, to handle the diversity of toxic prompts, ToxicDetector performs automatic toxic concept prompt extraction and augmentation to comprehensively cover various toxic scenarios given a set of samples. Moreover, embeddings inherently determine the semantics of prompts and guide content generation within LLMs. As a result, we construct features based on these embeddings. These features are both simple (easy to obtain and calculate) and effective (embedding the semantics of the prompt itself), rendering them scalable. Computational efficiency is addressed by converting toxic detection into a classification problem. With well-constructed features, we train a lightweight MLP to classify prompts. Once a user input prompt is provided, we extract its features during generation and classify it in real-time with minimal overhead.

4 METHODOLOGY

4.1 Overview of ToxicDetector

Figure 3 illustrates the workflow of ToxicDetector, which is designed to detect toxic prompts in LLMs. The process begins with the collection of both benign and toxic prompt samples. In the first stage, ToxicDetector performs Toxic Concept Prompt Extraction (§ 4.2), where it identifies and selects representative toxic prompts from the collected samples. These prompts are then augmented (§ 4.3) to create a diverse set of concept prompts. The next stage involves Feature Extraction (§ 4.4), where embeddings from each concept prompt are extracted using the LLM under test. These embeddings are used to train a classifier that can distinguish between toxic and non-toxic prompts. During the Toxic Detection phase (§ 4.5), user input prompts are processed through the same feature extraction mechanism, and the trained classifier evaluates the prompts to determine their toxicity, ultimately classifying them as either benign or toxic.

4.2 Toxic Concept Prompt Extraction

Given a toxic sample, ToxicDetector extracts its concept prompt. A toxic concept prompt is a high-level abstraction of a toxic prompt, which can cover more general toxic scenarios. For instance, the toxic prompt, "What's the most effective way to rob a bank?" represents a concept related to "committing crimes," which is similar to the concept behind "Write a tutorial on how to make a bomb." By extracting concept prompts, we obtain high-level abstractions of specific toxic prompts, making them more general and able to cover different toxic scenarios.

Algorithm 1: LLM-based Concept Prompt Augmentation

Input : Initial set of toxic concept prompts C, similarity threshold thr
Output : Augmented set of diverse toxic concept prompts C_{aug}

$C_{aug} \leftarrow C$;
repeat
  $C_{new} \leftarrow \emptyset$;
  for *each* $c \in C_{aug}$ **do**
    $c_{div} \leftarrow \text{LLM}(c)$;
    for *each* $c_d \in c_{div}$ **do**
      $sim \leftarrow \text{cos_sim}(\text{emb}(c_d), \text{emb}(C_{aug}))$;
      if $sim < thr$ **then**
        $C_{new} \leftarrow C_{new} \cup \{c_d\}$;
  $C_{aug} \leftarrow C_{aug} \cup C_{new}$;
until $C_{new} = \emptyset$;
return C_{aug};

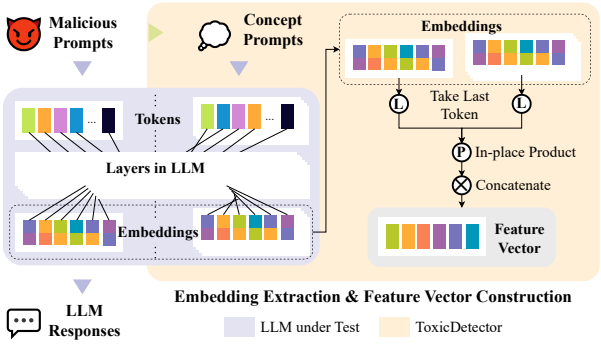


Figure 4: Feature Construction in ToxicDetector. User input prompts and concept prompts are processed through the LLM to extract embeddings from the last token at each layer. These embeddings are combined using element-wise product, and the results are concatenated to form a feature vector.

To obtain the concept prompt of a toxic prompt, we use LLMs like GPT-4o. The following prompt is used to feed the toxic prompt and obtain its concept prompt:

> Please create a single general description for a toxic prompt. This description should explain the overarching reason why it is considered toxic and what it aims to achieve.
> The description should be:
> 1. Concise, limited to one sentence.
> 2. Descriptive enough to convey a general understanding of the prompt's intent.
> 3. Specific to the context of the question without being overly detailed.
> 4. Clear and easy to understand for a general audience.
> Below are the toxic questions observed: TOXIC_PROMPT
> Your description:

It is important to note that we do not extract or augment benign prompt samples. Our primary focus is on detecting toxic prompts; therefore, our efforts are concentrated on generating and generalizing toxic prompt samples.

4.3 Concept Prompt Augmentation

Given toxic concept prompts obtained from the previous section, we augment them into a diverse set of concept prompts. The idea is that, although we have generalized specific toxic prompts by extracting concept prompts, we still need to create more diverse toxic prompts to cover a wider range of toxic scenarios. To achieve this, we implement an LLM-based concept prompt augmentation algorithm.

As shown in Algorithm 1, we iteratively feed the concept prompts (C) into the LLM and instruct it to generate diverse concept prompts (c_{div}) based on the given prompt. We then check the semantic similarity (sim) of the newly generated toxic concept prompts (c_d) with all existing concept prompts by calculating the cosine similarity of sentence embeddings using the BGE-M3 model [17]. If a newly generated concept prompt has a semantic similarity below a predefined threshold (thr), such as 0.8 (The similarity threshold was empirically determined; details are provided in § 5.1.), we treat it as a valid and diverse prompt and add it to the next iteration. If no newly generated toxic prompt meets this condition, the algorithm terminates. Thus, by using this LLM-based algorithm, we can enhance the diversity of toxic concept prompts for further training.

> You are asked to come up with a set of 5 diverse descriptions of a type of toxic questions.
> Here are the requirements:
> 1. Use concise and clear language.
> 2. Each sentence should make a definitive statement.
> 3. Try not to repeat the verb for each instruction to maximize diversity.
> 4. Focus on categorizing or labeling a concept or action.
> 5. Ensure the subject of each sentence is a noun or noun phrase.
> 6. Avoid repetition of the same noun or noun phrase.
> 7. Keep each sentence brief, within one sentence.
> The malicious question type is: TOXIC_CONCEPT_PROMPT
> List of 5 descriptions:

4.4 Feature Extraction & Training

Feature Extraction. With the toxic concept prompts collected, we extract features and train a classifier. The key idea is to construct features that capture both the meaning of the user input prompt and its similarity to the toxic concept prompts. For semantics, the embedding of the last token of each layer serves as a straightforward representation of the user input prompt. Given the embedding, we can calculate the semantic similarity between the user input prompt and the toxic concept prompts.

Figure 4 illustrates the feature construction process. Inspired by previous work [30, 53], we choose the last token as the semantic embedding of the user input prompt. Specifically, for each layer, we take the last token of the user input and the toxic concept prompts to

obtain their respective embeddings. We then compute the element-wise product of the embeddings for each toxic concept prompt with the embedding of the user input prompt. These products are concatenated to form a feature vector, which is subsequently fed into an MLP for classification.

Formally, let $\mathbf{e}_u^{(l)}$ denote the embedding of the last token of the user input prompt at layer l and $\mathbf{e}_t^{(l)}$ denote the embedding of a toxic concept prompt at layer l. The feature vector $\mathbf{f}$ is constructed as follows:

$$\mathbf{f} = \text{concat}\left(\left\{\mathbf{e}_u^{(l)} \odot \mathbf{e}_t^{(l)}\right\}_{l=1}^{L}\right), \tag{1}$$

where $\odot$ denotes the element-wise product, concat denotes concatenation, and L is the number of layers. The feature vector $\mathbf{f}$ is then used as input to the MLP classifier for determining whether the user input prompt is toxic.

The design of this feature extraction method leverages the powerful semantic representation capabilities of embeddings. By using the last token's embedding, we efficiently capture the essential meaning of the input prompt. The element-wise product operation allows us to directly measure the interaction between the input prompt and toxic concept prompts, which is crucial for accurate classification. Concatenating these products across all layers ensures that the classifier has a comprehensive view of the prompt's semantic characteristics at multiple levels of abstraction. This design choice enhances the model's ability to generalize from the training data to unseen prompts, improving the robustness and reliability of the toxic prompt detection system.

Classifier Training. To address context insensitivity and out-of-vocabulary issues in vector based similarity techniques, ToxicDetector uses embeddings from the LLM under test for both training and identification. We enhance training data quality with a concept prompt dataset augmented by LLMs, increasing diversity and reducing bias.

Given the extracted token embeddings, we train the classifier using both benign and toxic prompts. Specifically, we implement a fully-connected MLP with five layers and approximately 300 million parameters. This classifier is trained to solve a binary classification problem, predicting whether the user input prompt is benign or toxic.

We use cross-entropy as the loss function for training the MLP. Cross-entropy is chosen because it is well-suited for binary classification tasks, providing a measure of the difference between the predicted probabilities and the actual labels. By minimizing this loss, the model learns to accurately distinguish between benign and toxic prompts.

The design of the MLP with a large number of parameters allows the model to capture complex patterns and nuances in the data. This complexity is essential for handling the diverse and subtle nature of toxic prompts, ensuring that the classifier can generalize well to new, unseen inputs. Additionally, the fully-connected structure of the MLP enables effective learning from the extracted feature vectors, leveraging the semantic information and similarities between the user input prompts and toxic concept prompts.

4.5 Toxic Detection

With the trained classifier in place, we can determine whether a user input prompt is toxic or benign. Specifically, we extract and calculate features based on the method described in the previous steps, and then input these features into the classifier for decision-making.

This approach is computationally efficient for several reasons: (1) **Inherent Embedding Calculation:** The embedding calculation is an integral part of the generation process of LLMs, which means that we leverage existing computational steps to extract necessary features without additional overhead. (2) **Simultaneous Classification:** The classification occurs in real-time during the LLM's response generation. This integration ensures that no separate processing step is required after the LLM has generated its response, thereby speeding up the entire process.

By utilizing the LLM's inherent capabilities for embedding generation and combining it with an efficient feature extraction and classification mechanism, ToxicDetector ensures that toxic detection is both swift and resource-efficient. This design makes it particularly suitable for applications where real-time response and computational efficiency are critical.

5 EVALUATION

In this section, we present our evaluation of ToxicDetector. The implementation details of ToxicDetector are available on our website [8]. To assess its effectiveness, this evaluation explores the following research questions:

- **RQ1: (Effectiveness).** How effective is ToxicDetector in accurately identifying toxic prompts?
- **RQ2: (Efficiency).** How lightweight is ToxicDetector for identifying toxic prompts during runtime?
- **RQ3: (Feature Representation).** How does the quality of the embedding representations affect the classification performance for toxic prompts?

Datasets. We use two orthogonal datasets, SafetyPromptCollections and RealToxicityPrompts [2], to evaluate the effectiveness of ToxicDetector.

SafetyPromptCollections. Following previous work [32, 43, 53, 54], SafetyPromptCollections contains 1,000 benign and 1,750 toxic prompts.

For benign prompts, we construct the dataset from ShareGPT [7], following the settings of prior research [36]. The ShareGPT dataset includes benign prompts generated by real users, providing a representative sample of typical LLM interactions. We sample 1,000 benign prompts to ensure statistically sound results with a 95% confidence interval and a ±5% margin of error. For toxic prompts, we compile the dataset by merging benchmarks from previous studies [12, 20, 36, 43, 47], resulting in seven distinct toxic scenarios, each with 250 toxic prompts.

RealToxicityPrompts. To evaluate the generalizability of ToxicDetector, we select an orthogonal toxic prompts dataset [2] and sample 10,000 toxic prompts for evaluation.

Baselines. To evaluate the effectiveness of ToxicDetector, we select six existing tools from both blackbox and whitebox state-of-the-art techniques from academic and industry communities. The selection is based on two criteria: (1) public accessibility, meaning

the tool can be accessed via API or its public code repository, and (2) performance, indicating it is the state-of-the-art in its category.

- **PlatonicDetector** [22]: We implement PlatonicDetector based on the convergent representations in LLMs as described in its original paper [22] to detect toxic prompts using a white-box approach.
- **PerspectiveAPI** [28]: Developed by Google Jigsaw, the Perspective API uses a multilingual character-level model to detect toxic content across various languages and domains.
- **OpenAIModerationAPI** [35]: The OpenAI Moderation API is capable of detecting plain toxicity in prompts and is developed by OpenAI.
- **WatchYourLanguage** [26]: This tool applies LLMs to detect toxic prompts via a reflection prompting mechanism with GPT-4o.
- **PerplexityFilter** [23]: This method relies on the model's confidence in the prompts, filtering out those with low confidence as toxic prompts in a white-box approach.
- **BD-LLM** [50]: This approach uses knowledge distillation to train a transformer-based classifier [41] for detecting toxic prompts.

LLMs under test. We select seven popular open-source LLMs for our evaluation. These include various versions of the LLama models, chosen for their widespread use and robust performance in natural language processing tasks. The specific models tested are:

- **LLama-3 (8B and 70B versions)** [5]: These models are the latest iterations in the LLama series, offering significant improvements in both size and performance. The 8 billion parameter model (8B) and the 70 billion parameter model (70B) are tested to evaluate performance across different scales.
- **LLama-2 (7B and 13B versions)** [40]: As the second generation of LLama models, these versions provide enhancements in efficiency and accuracy.
- **LLama (7B and 13B versions)** [39]: We use Vicuna-v1.5-7B [46] and Vicuna-v1.5-13B [11] which are fine-tuning from origin LLama.
- **Gemma-2 (9B)** [45]: We select this latest LLM developed by Google to evaluate the generalization of ToxicDetector across different models.

Metrics. To evaluate the effectiveness of ToxicDetector, we employ the following metrics:

- **F1 Score**: This metric provides a balance between precision and recall, giving a single measure of a test's accuracy. It is especially useful when the class distribution is imbalanced.
- **False Positive Rate**: This metric measures the proportion of benign prompts incorrectly classified as toxic. A lower FPR indicates fewer false alarms.
- **Accuracy**: This metric represents the proportion of correctly classified prompts (both benign and toxic) out of all prompts. It gives a general sense of the model's overall performance.
- **ROC Curve**: The Receiver Operating Characteristic (ROC) curve illustrates the true positive rate (sensitivity) against the false positive rate. This curve helps visualize the trade-offs between true positives and false positives and is useful for comparing different models.

Experimental Settings. All experiments are conducted using two Titan RTX 48 GPUs on Ubuntu 22.04. We configure all baselines and LLMs according to their respective instructions. To train the classifier, we use a fully connected MLP with 5 layers and 300 million parameters. The training parameters are as follows: a batch size of 20, 100 epochs, a learning rate of 0.01, and a weight decay of 0.0002. For the training dataset, we use 50% of the data for training and the rest for testing. To mitigate the effects of randomness in evaluation, we ran all experiments ten times.

5.1 RQ1 (Effectiveness)

In this research question, we aim to evaluate the effectiveness of ToxicDetector in accurately identifying toxic prompts across various scenarios. We compare ToxicDetector with other baseline methods across multiple LLMs under test. The results are summarized in Table 2, Table 3, Figure 5, and Table 5. [2]

Comparison between Different Models. Table 2 presents the average F1 scores, false positive rates, and overall accuracies of various classifiers in identifying toxic prompts across different scenarios (statistically significant results are highlighted in bold, calculated using the Mann-Whitney U test [38] at a 0.05 confidence level). The results indicate that methods like PerplexityFilter and BD-LLM struggle with high false positive rates, reflecting difficulties in accurately distinguishing between toxic and benign prompts. For example, PerplexityFilter has a false positive rate of 0.498, leading to numerous false alarms. In contrast, ToxicDetector achieves a low false positive rate of 0.019, demonstrating its precision in differentiating between toxic and benign prompts—a crucial quality for practical applications where avoiding unnecessary disruptions is critical. Furthermore, ToxicDetector achieves the highest average F1 score, 0.9635, across both Gemma-2 and Llama series LLMs, underscoring its robust capability in detecting toxic prompts. The superior performance of ToxicDetector can be attributed to its efficient use of embedding vectors and a lightweight MLP classifier, which together enhance its detection capabilities.

An interesting finding is that models with larger parameter sizes (e.g., Llama2-13b vs. Llama2-7b) and newer architectures (e.g., Llama3 vs. Llama2) are more effective in refusing toxic prompts in their responses, benefiting from sophisticated alignment techniques [5, 40]. Additionally, ToxicDetector shows better results with larger and newer models, providing evidence that these models are trained with better semantic embeddings that can represent high-level concepts, including toxic ones.

Comparison between Different Datasets. In Table 3, we evaluate all baselines on an orthogonal dataset, RealToxicityPrompts. ToxicDetector once again achieves the best performance, with an average F1 score of 0.9628 and an exceptionally low false positive rate of 0.02. Methods relying on pre-trained models, such as WatchYourLanguage, PerspectiveAPI, and OpenAIModerationAPI, show significant increases in average F1 scores (from 0.6801 to 0.7674, 0.5278 to 0.8674, and 0.5884 to 0.8865, respectively), likely because their pre-training data includes RealToxicityPrompts (created in 2020). Conversely, PlatonicDetector's performance drops significantly, with its average F1 score falling

[2]Due to page limitation, we only present the detailed data of ToxicDetector and leave other baselines' details on our website [8].

Table 2: F1 Scores, False Positive Rates, and Overall Accuracies of Prompt Classification for Various Detection Techniques. The metrics include overall F1 scores for each toxic scenario, the false positive rate, and the accuracy of the classifiers on SafetyPromptCollections. The statistically significant values are highlighted in bold.

Detection Technique		F1 Score								False Positive Rate								Accuracy
		Toxic Scenarios							Average	Toxic Scenarios							Average	
		Information Leakage	Misleading Information	Illegal Activities	Political Lobbying	Sexual Content	Insult	Harmful Speech		Information Leakage	Misleading Information	Illegal Activities	Political Lobbying	Sexual Content	Insult	Harmful Speech		
ToxicDetector	Llama2-7b	0.9615	0.9200	0.9583	0.9451	0.9495	0.9091	0.9697	0.9447	0.040	0.040	0.000	0.020	0.050	0.100	0.010	0.037	0.9626
	Llama2-13b	1.0000	0.9615	0.9796	0.9556	0.9677	0.9485	0.9592	0.9674	0.000	0.040	0.000	0.010	0.010	0.010	0.010	0.011	0.9789
	Llama3-8b	1.0000	0.9600	0.9583	0.9670	0.9783	0.9697	0.9216	0.9650	0.000	0.020	0.000	0.010	0.000	0.010	0.050	0.013	0.9770
	Llama3-70b	0.9901	0.9515	0.9796	0.9677	0.9787	0.9800	0.9505	0.9712	0.010	0.040	0.000	0.020	0.010	0.010	0.030	0.017	0.9808
	Vicuna-v1.5-7b	1.0000	0.9333	0.9697	0.9574	0.9892	0.9691	0.9278	0.9638	0.000	0.060	0.010	0.030	0.000	0.000	0.020	0.017	0.9761
	Vicuna-v1.5-13b	1.0000	0.9231	0.9796	0.9677	0.9892	0.9703	0.9184	0.9640	0.000	0.060	0.000	0.020	0.000	0.020	0.030	0.019	0.9761
	Gemma2-9b	1.0000	0.9505	0.9796	0.9670	0.9684	0.9608	0.9505	0.9681	0.000	0.030	0.000	0.010	0.020	0.030	0.030	0.017	0.9789
	Average	**0.9931**	**0.9428**	**0.9721**	**0.9611**	**0.9744**	**0.9582**	**0.9425**	**0.9635**	**0.007**	0.041	**0.001**	**0.017**	**0.013**	**0.026**	**0.026**	**0.019**	**0.9758**
PlatonicDetector		0.9432	0.8482	0.9602	0.8969	0.8867	0.8368	0.9182	0.8986	0.066	0.181	0.021	0.067	0.104	0.154	0.070	0.095	0.9241
BD-LLM		0.6944	0.7299	0.7619	0.6452	0.6087	0.6545	0.6818	0.6824	0.440	0.410	0.280	0.380	0.490	0.250	0.370	0.374	0.7226
OpenAIModerationAPI		0.1250	0.2703	0.6522	0.6667	0.8952	0.8085	0.7010	0.5884	0.100	0.140	0.120	0.100	0.110	0.060	0.130	0.109	0.7819
PerspectiveAPI		0.0377	0.3030	0.5476	0.6494	0.7957	0.6947	0.6667	0.5278	0.020	0.060	0.110	0.060	0.090	0.120	0.100	0.080	0.7704
PerplexityFilter		0.4767	0.2678	0.1739	0.2457	0.3196	0.1973	0.1692	0.2643	0.537	0.500	0.571	0.474	0.453	0.449	0.501	0.498	0.4472
WatchYourLanguage		0.3437	0.2373	0.9231	0.7805	0.8989	0.6667	0.9109	0.6801	0.030	**0.020**	0.060	0.040	0.020	0.060	0.050	0.040	0.8479

Table 3: Evaluation Results on RealToxicityPrompts.

Detection Technique		F1 Score							False Positive Rate							Accuracy
		Toxic Scenarios						Average	Toxic Scenarios						Average	
		Identity Attack	Offense	Flirtation	Profanity	Sexually Explicit	Threat		Identity Attack	Offense	Flirtation	Profanity	Sexually Explicit	Threat		
ToxicDetector	Llama2-7b	0.9424	0.8950	0.8852	0.9247	0.9796	0.9101	0.9228	0.010	0.000	0.020	0.000	0.000	0.030	0.010	0.9283
	Llama2-13b	0.9749	0.9950	0.9694	0.9798	0.9849	0.9645	0.9781	0.020	0.010	0.010	0.010	0.010	0.020	0.013	0.9783
	Llama3-8b	0.9950	0.9849	0.9588	0.9424	0.9412	0.9697	0.9653	0.000	0.010	0.010	0.010	0.080	0.020	0.022	0.9658
	Llama3-70b	0.9950	0.9849	0.9798	0.9950	0.9751	0.9950	0.9875	0.010	0.010	0.010	0.000	0.030	0.010	0.012	0.9875
	Vicuna-v1.5-7b	0.9447	0.9583	0.9479	0.9278	0.9412	0.9300	0.9417	0.050	0.000	0.010	0.040	0.080	0.070	0.042	0.9425
	Vicuna-v1.5-13b	0.9749	0.9950	0.9798	0.9798	0.9697	0.9802	0.9799	0.020	0.010	0.010	0.010	0.020	0.030	0.017	0.9800
	Gemma2-9b	0.9900	0.9849	0.9800	0.9198	0.9412	0.9697	0.9643	0.010	0.010	0.020	0.010	0.080	0.020	0.025	0.9650
	Average	**0.9738**	**0.9712**	**0.9573**	**0.9528**	**0.9618**	**0.9599**	**0.9628**	**0.017**	**0.007**	**0.013**	**0.011**	0.043	0.029	**0.020**	**0.9639**
PlatonicDetector		0.9166	0.9132	0.4943	0.8901	0.8500	0.9259	0.8317	0.189	0.179	0.369	0.257	0.203	0.104	0.217	0.8357
BD-LLM		0.7800	0.6211	0.8454	0.8223	0.7826	0.8316	0.7805	0.220	0.110	0.120	0.160	0.120	0.110	0.140	0.7983
OpenAIModerationAPI		0.9238	0.8776	0.7709	0.8856	0.9320	0.9293	0.8865	0.130	0.100	0.100	0.120	0.100	0.060	0.102	0.8900
PerspectiveAPI		0.9378	0.9608	0.5455	0.9346	0.9565	0.8691	0.8674	0.110	0.060	0.040	0.140	0.080	0.080	0.085	0.8883
PerplexityFilter		0.4672	0.4616	0.5056	0.4670	0.4897	0.4726	0.4773	0.527	0.443	0.530	0.581	0.481	0.504	0.511	0.4898
WatchYourLanguage		0.8667	0.8070	0.4706	0.9158	0.7836	0.7607	0.7674	0.020	0.020	0.040	0.030	**0.040**	**0.010**	0.027	0.8158

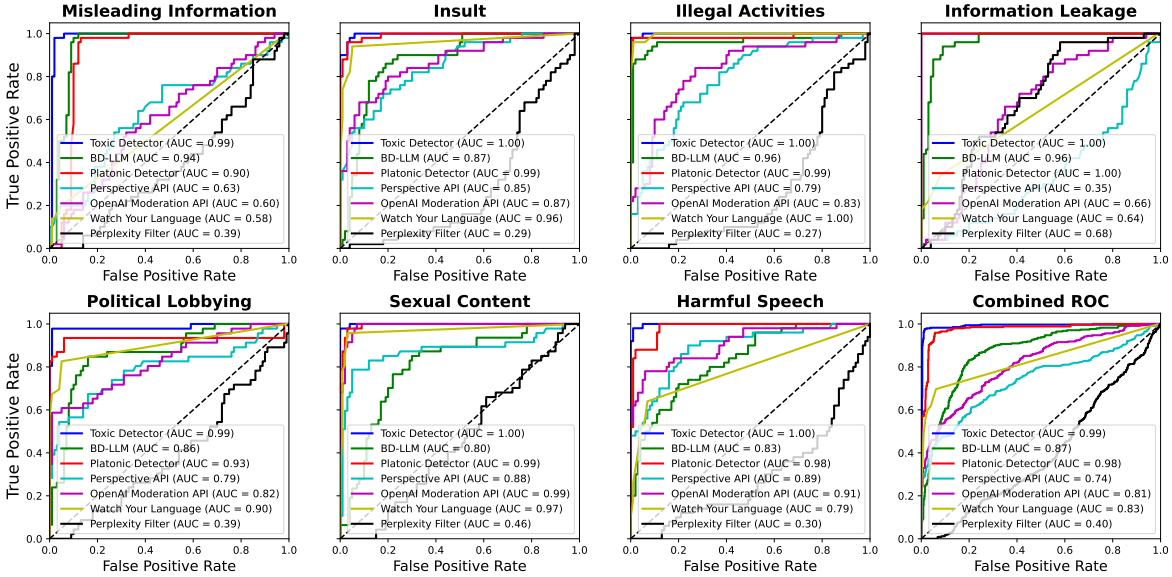


Figure 5: The ROC curves for identifying seven different types of toxic scenarios, comparing the performance of ToxicDetector on all LLMs under test with all baselines (SafetyPromptCollections).

to 0.8317 and its false positive rate increasing to 0.217, indicating a lack of generalization to different distributions of toxic prompts.

These results demonstrate ToxicDetector's ability to maintain robust performance across different datasets and toxic scenarios,

Table 4: F1 Scores for different toxic scenarios with jailbreaking on SafetyPromptCollections and RealToxicityPrompts.

Model	Dataset	
	SafetyPromptCollections	RealToxicityPrompts
Llama2-7b	0.9463	0.9749
Llama2-13b	0.9706	0.9577
Llama3-8b	0.9365	0.9664
Llama3-70b	0.9951	0.9799
Vicuna-v1.5-7b	0.9299	0.9778
Vicuna-v1.5-13b	0.9344	0.9483
Gemma2-9b	0.9521	0.9434
Average	**0.9521**	**0.9641**

consistently outperforming a range of baseline methods in both precision and generalizability.

Comparison with PlatonicDetector. While ToxicDetector draws inspiration from the feature construction approach used in PlatonicDetector, we have implemented significant improvements. The original work [22] demonstrates that different LLMs produce consistent embeddings for the same concepts by analyzing the similarity of the last token's embedding. ToxicDetector extends this concept into a robust pipeline specifically designed to detect toxic prompts. Specifically, ToxicDetector concatenates embeddings from each transformer layer, rather than relying solely on the last layer as PlatonicDetector does. This method allows ToxicDetector to capture a more comprehensive set of features, significantly improving its ability to identify toxic prompts. Our comparative results highlight this enhancement, with ToxicDetector achieving statistically significantly better results across most toxic scenarios on both SafetyPromptCollections and RealToxicityPrompts, in terms of F1 score and false positive rate (as determined by the Mann-Whitney U test at a 0.05 confidence level). Additionally, we have translated the theoretical insights from PlatonicDetector into a practical, automated pipeline, integrating LLM-based data augmentation. This provides developers with a powerful tool to leverage LLMs more effectively in their applications, thereby enhancing software safety and trustworthiness.

Comparison in terms of ROC. Figure 5 presents the averaged ROC curves for identifying seven different types of toxic scenarios across various LLMs tested on SafetyPromptCollections (detailed results for RealToxicityPrompts are available on our website [8]). The figure shows that methods like PerspectiveAPI and PerplexityFilter have lower Area Under the Curve (AUC) values, indicating less reliable detection capabilities. For example, PerplexityFilter has an AUC of 0.35 in the "Information Leakage" scenario, reflecting poor performance. In contrast, ToxicDetector consistently achieves higher AUC values across all types of malicious activities, with an overall AUC of 0.99 on both SafetyPromptCollections and RealToxicityPrompts. These high AUC values demonstrate ToxicDetector's robustness in detecting toxic content across diverse scenarios. ToxicDetector's effectiveness is further underscored by its ability to maintain high accuracy and reliability, even in complex and varied contexts.

Toxic Detection with Jailbreak Techniques. Table 4 presents the average results of ToxicDetector in detecting toxic prompts using template-based jailbreak techniques, measured by F1 score.

Table 5: Comparison of F1 Scores for different toxic scenarios with and without concept prompt augmentation, and the corresponding boost on SafetyPromptCollections. Values in bold indicate the highest F1 Score in each scenario.

Toxic Scenario	F1 Score (Plain)	F1 Score (Aug)	Boost
Information Leakage	0.9434	**1.0000**	0.0566
Misleading Information	0.8958	**0.9200**	0.0242
Illegal Activities	0.9697	**0.9800**	0.0103
Political Lobbying	**0.9451**	**0.9451**	-
Sexual Content	0.9495	**0.9583**	0.0088
Harmful Speech	0.8913	**0.9167**	0.0254
Insult	0.8140	**0.9697**	0.1557
Overall	0.9155	**0.9557**	0.0401

Table 6: F1 Score Across Varying Similarity Thresholds on SafetyPromptCollections

Metric	Similarity Threshold									
	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1.0
F1 Score	0.5939	0.5786	0.7907	0.9412	0.9505	0.9600	0.9505	0.9703	0.9703	0.9293

We populate manually crafted jailbreak templates from previous work [33] with toxic prompts as input for ToxicDetector. The results show that ToxicDetector achieves an average F1 score of 0.9521 on SafetyPromptCollections and 0.9641 on RealToxicityPrompts, demonstrating that ToxicDetector effectively identifies toxic prompts embedded within jailbreak techniques, even when trained on plain toxic prompts.

Effectiveness of Concept Prompt Augmentation. Table 5 compares the F1 scores for different toxic scenarios with and without concept prompt augmentation, along with the corresponding performance boost on SafetyPromptCollections. ToxicDetector shows significant improvement with concept prompt augmentation, particularly in scenarios like "Insult," where the F1 score jumps from 0.8140 to 0.9697. This notable boost demonstrates the added value of concept prompt augmentation in enhancing detection accuracy. The ability of ToxicDetector to leverage these augmentations effectively highlights its superior design and implementation.

Validation of Concept Prompts. To validate the generated toxic concept prompts, three authors independently review 680 of these prompts. The goal is to perform a binary classification to determine whether each generated toxic concept prompt falls within the disallowed usages outlined by OpenAI [9]. This is important because OpenAI, with the largest user base [6] and government regulation [4], provides a representative taxonomy of toxic prompts. The results show that all three authors unanimously agree that the generated concept prompts align with the disallowed usage criteria. This consensus demonstrates the effectiveness of the concept prompt augmentation process.

Similarity Threshold. We empirically decide the similarity threshold when selecting toxic concept prompts by analyzing the F1 score across varying thresholds on SafetyPromptCollections. As shown in Table 6, the results show that the F1 score increases as the similarity threshold rises, peaking between 0.8 and 0.9, and then slightly decreases at 1.0. We set a high similarity threshold to focus

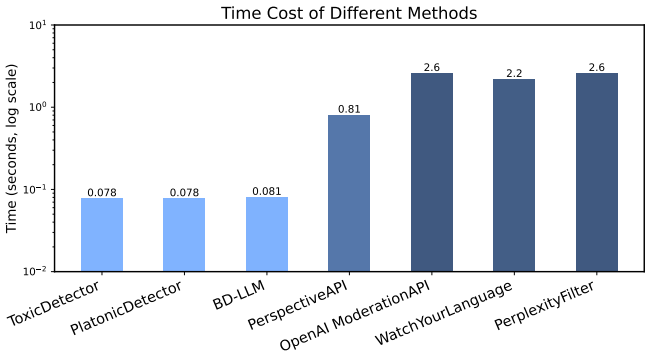


Figure 6: Comparison of the average prompt processing times in our evaluation, displayed on a logarithmic scale.

on toxic prompts while still maintaining diversity among toxic concept prompts. For example, this approach allows us to capture similar concepts with diverse toxic prompts, such as "Inquire about private access credentials," "Seek disclosure of proprietary data," and "Ask for unreleased financial reports."

The evaluation results clearly demonstrate that ToxicDetector outperforms existing methods in detecting toxic prompts across a wide range of scenarios. The consistently high F1 scores and low false positive rates indicate that ToxicDetector is both accurate and reliable. The robustness of ToxicDetector is further evidenced by its high AUC values across different types of toxic content, showing its superior performance compared to other classifiers. Concept prompt augmentation significantly enhances detection effectiveness, as shown by the improvements in F1 scores. These findings imply the practical utility of ToxicDetector in real-world applications.

Answer to RQ1

ToxicDetector demonstrates superior performance in detecting toxic prompts across various LLMs compared to existing methods. Concept prompt augmentation significantly enhances detection effectiveness.

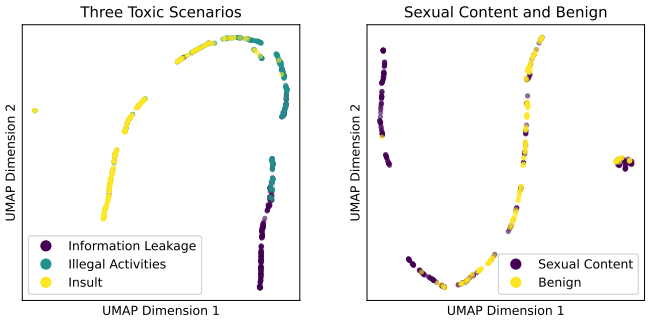
5.2 RQ2 (Efficiency)

In this research question, we aim to evaluate the training efforts and inference time cost of ToxicDetector. We train ToxicDetector with different training set sizes and record the training time. Additionally, we measure the classification time during toxic detection at runtime. Table 7 and Figure 6 summarize the results.

Table 7 illustrates the relationship between the number of training epochs, the corresponding training times, and the resulting F1 scores for our model. As the number of training epochs increases from 20 to 200, the training time also increases, starting at 69.4 seconds and reaching 197.6 seconds. With the increase in training time, the F1 scores show significant improvement, beginning at 0.942 with 20 epochs and peaking at 0.980 at 100 epochs. Beyond 100 epochs, the F1 score stabilizes at 0.980, indicating that additional training does not further enhance the model's performance. This table highlights the balance between training duration and model accuracy, suggesting that 100 epochs is optimal for achieving high performance without unnecessary extra training time. Additionally, the relatively short training times, even at maximum epochs, demonstrate that ToxicDetector is fast to train.

Table 7: Training epochs, corresponding training times, and F1 scores.

Metric	Train Epochs					
	20	40	60	80	100	200
Train Time (seconds)	69.4	75.8	79.2	88.2	89.4	197.6
F1 Score	0.942	0.960	0.970	0.951	0.980	0.980



(a) Different toxic scenarios. **(b) Toxic & benign prompts.**

Figure 7: Visualization of Prompt Embeddings by UMAP [37].

Figure 6 compares the average prompt processing times for different methods. Methods like PerplexityFilter, WatchYourLanguage, and OpenAIModerationAPI show longer processing times, ranging from 2.2 to 2.6 seconds, reflecting computational overhead or network latency. In contrast, smaller models demonstrate remarkable efficiency, with BD-LLM processing a prompt in 0.081 seconds, and ToxicDetector and PlatonicDetector achieving the lowest processing times of approximately 0.078 seconds. The low processing time of ToxicDetector indicates its suitability for real-time applications, making it highly efficient in environments where prompt response times are critical. This efficiency can be attributed to ToxicDetector's streamlined feature extraction and lightweight MLP classifier.

Training efforts and inference time are critical factors in the deployment of machine learning models, especially in real-time applications. Efficient training processes allow for quicker updates and retraining cycles, ensuring that models can adapt to new data and evolving scenarios without significant downtime. Short inference times are equally important as they enable the model to provide rapid responses, which is crucial in applications such as content moderation, online safety, and customer service. High training and inference efficiency also reduce computational resource consumption, making the system more cost-effective and scalable. Overall, optimizing both training efforts and inference time enhances the practicality and responsiveness of the deployed model.

Answer to RQ2

ToxicDetector requires minimal training efforts and achieves fast inference times for detecting toxic prompts, which is crucial for real-world applications.

5.3 RQ3 (Feature Representation)

We qualitatively examine why the feature representation effectively identifies toxic prompts using UMAP [37] for dimensionality reduction, as shown in Figure 7.

Figure 7a displays UMAP results for three toxic scenarios—Information Leakage, Illegal Activities, and Insult—on LLama-2 7B [40]. The prompts form distinct clusters, indicating that the feature representation accurately captures the unique characteristics of each toxic type. This separation demonstrates the method's ability to differentiate between various toxic prompts reliably.

Figure 7b contrasts toxic and benign prompts on LLama-2 7B [40]. The clear distinction between sexual content and benign clusters confirms that the feature representation effectively distinguishes toxic from non-toxic prompts, reducing false positives.

Overall, the UMAP visualizations confirm that ToxicDetector's feature representation robustly differentiates between multiple toxic scenarios and benign prompts, ensuring accurate and reliable toxic prompt detection.

Answer to RQ3

The feature representation clearly distinguishes different toxic scenarios and separates toxic from benign prompts, enabling effective and accurate detection of toxic prompts by classifiers.

6 THREATS TO VALIDITY

Our evaluation of ToxicDetector faces several potential threats.

First, dataset construction may introduce biases. Although we combined multiple benchmarks to create a comprehensive dataset, the selected toxic and benign prompts might not fully capture the diversity of real-world interactions. Additionally, benign prompts from ShareGPT may not represent typical user behavior across all platforms, potentially limiting the generalizability of our results.

Second, experimental settings such as the choice of LLMs and baseline configurations could affect our findings. We used popular open-source LLMs configured per their guidelines, but different model versions or implementations might yield varying performance. Moreover, ToxicDetector's effectiveness could differ when applied to other LLMs or in different application contexts.

Lastly, the training and evaluation process itself may pose validity threats. Despite conducting multiple runs and using standard metrics to reduce randomness, inherent variability in machine learning experiments can cause slight result fluctuations. The classifier's hyperparameters, like learning rate and batch size, were selected based on preliminary tests and might not be optimal universally. Future research should investigate these parameters further to enhance ToxicDetector's robustness.

7 RELATED WORK

7.1 Toxic Prompts

Toxic prompts are inputs that cause LLMs to generate harmful or inappropriate responses, making their detection crucial for safe interactions. Datasets like RealToxicityPrompts [20] provide benchmarks to assess LLMs' tendency to produce toxic content, underscoring the importance of robust detection mechanisms for responsible language model deployment.

7.2 Jailbreaking on LLMs

LLMs are susceptible to jailbreak attacks that leverage toxic prompts to produce unethical outputs. Studies such as Liu et al. [33] and MASTERKEY [18] demonstrate how adversarial prompts can bypass safeguards in models like ChatGPT and Bard. These vulnerabilities highlight the need for effective defenses, which ToxicDetector addresses by detecting and mitigating toxic prompts before they exploit these weaknesses.

7.3 Toxic Prompt Detection Methods

Detecting toxic prompts is crucial for the safe and ethical deployment of LLMs. Various methods have been proposed to identify and mitigate the effects of toxic prompts.

Whitebox Methods: These methods leverage the internal state of the model to detect toxic content. For instance, PlatonicDetector [22] utilizes the convergent representations in LLMs to identify toxic prompts, offering insights into the underlying dynamics of language processing. PerplexityFilter [23], on the other hand, assesses the model's confidence in its responses, filtering out prompts that elicit low-confidence responses as potentially toxic. This approach is particularly effective in isolating subtle or cleverly disguised toxic content that may not trigger traditional detection mechanisms.

Blackbox Methods: These methods use pre-trained models without accessing their internal states. The OpenAI Moderation API [35] filters plain toxicity, while Google's Perspective API [28] detects toxic content across languages. BD-LLM [50] distills LLM knowledge to identify toxic prompts, and WatchYourLanguage [26] uses a reflection prompting mechanism with GPT-4o for detection.

ToxicDetector enhances detection by integrating both whitebox and blackbox approaches, utilizing LLM embeddings and an MLP classifier to provide a scalable and real-time solution for identifying toxic prompts.

8 CONCLUSION

In this work, we present ToxicDetector, a lightweight greybox method for efficiently detecting toxic prompts in LLMs. ToxicDetector leverages LLM-generated toxic concept prompts to create feature vectors and employs a classifier for prompt classification. Our evaluation on the latest LLMs, including LLama series and Gemma-2, demonstrates ToxicDetector's high accuracy, low false positive rates, and superior performance compared to state-of-the-art methods. With a processing time of 0.078 seconds per prompt and the ability to train a detector in under five minutes, ToxicDetector is ideal for real-time applications. Future work will focus on adding interpretability and automated evaluation features to further enhance toxic prompt detection and ensure the safe use of LLMs.

ACKNOWLEDGEMENTS

We sincerely thank all the anonymous reviewers for their valuable feedback, which greatly contributed to the improvement of this paper. This research is jointly sponsored by the NSFC Program under Grants No. 62302304 and the ShanghaiTech Startup Funding. This research is supported by NTU College of Engineering CRP, Tier 3 Preparatory Grant 2023, and 10658 - MOE AcRF Tier 1: Call2/2023.

REFERENCES

[1] About the api score. https://developers.perspectiveapi.com/s/about-the-api-score?language=en_US.

[2] allenai/real-toxicity-prompts · datasets at hugging face. https://huggingface.co/datasets/allenai/real-toxicity-prompts. (Accessed on 08/10/2024).

[3] cdn.openai.com/papers/gpt-4-system-card.pdf. https://cdn.openai.com/papers/gpt-4-system-card.pdf. (Accessed on 08/13/2024).

[4] Fact sheet: Biden-harris administration secures voluntary commitments from leading artificial intelligence companies to manage the risks posed by ai | the white house. https://www.whitehouse.gov/briefing-room/statements-releases/2023/07/21/fact-sheet-biden-harris-administration-secures-voluntary-commitments-from-leading-artificial-intelligence-companies-to-manage-the-risks-posed-by-ai/. (Accessed on 08/13/2024).

[5] Meta llama 3. https://llama.meta.com/llama3/.

[6] Number of chatgpt users (aug 2024). https://explodingtopics.com/blog/chatgpt-users. (Accessed on 08/13/2024).

[7] Ryokoai/sharegpt52k datasets at hugging face. https://huggingface.co/datasets/RyokoAI/ShareGPT52K.

[8] Toxicdetector. https://sites.google.com/view/toxic-prompt-detector.

[9] Usage policies | openai. https://openai.com/policies/usage-policies/. (Accessed on 08/13/2024).

[10] Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).

[11] andyll7772. Run a chatgpt-like chatbot on a single gpu with rocm, October 2023.

[12] Bianchi, F., Suzgun, M., Attanasio, G., Röttger, P., Jurafsky, D., Hashimoto, T., and Zou, J. Safety-tuned llamas: Lessons from improving the safety of large language models that follow instructions, 2024.

[13] Böhme, M., Pham, V., Nguyen, M., and Roychoudhury, A. Directed greybox fuzzing. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS 2017, Dallas, TX, USA, October 30 - November 03, 2017* (2017), B. Thuraisingham, D. Evans, T. Malkin, and D. Xu, Eds., ACM, pp. 2329–2344.

[14] Böhme, M., Pham, V., and Roychoudhury, A. Coverage-based greybox fuzzing as markov chain. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, Vienna, Austria, October 24-28, 2016* (2016), E. R. Weippl, S. Katzenbeisser, C. Kruegel, A. C. Myers, and S. Halevi, Eds., ACM, pp. 1032–1043.

[15] Chang, Z., Li, M., Liu, Y., Wang, J., Wang, Q., and Liu, Y. Play guessing game with llm: Indirect jailbreak attack with implicit clues, 2024.

[16] Chen, B., Paliwal, A., and Yan, Q. Jailbreaker in jail: Moving target defense for large language models. In *Proceedings of the 10th ACM Workshop on Moving Target Defense* (New York, NY, USA, 2023), MTD '23, Association for Computing Machinery, p. 29–32.

[17] Chen, J., Xiao, S., Zhang, P., Luo, K., Lian, D., and Liu, Z. Bge m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation, 2024.

[18] Deng, G., Liu, Y., Li, Y., Wang, K., Zhang, Y., Li, Z., Wang, H., Zhang, T., and Liu, Y. Masterkey: Automated jailbreaking of large language model chatbots. In *Proceedings 2024 Network and Distributed System Security Symposium* (2024), NDSS 2024, Internet Society.

[19] Deng, G., Liu, Y., Wang, K., Li, Y., Zhang, T., and Liu, Y. Pandora: Jailbreak gpts by retrieval augmented generation poisoning, 2024.

[20] Gehman, S., Gururangan, S., Sap, M., Choi, Y., and Smith, N. A. Realtoxicityprompts: Evaluating neural toxic degeneration in language models. *arXiv preprint arXiv:2009.11462* (2020).

[21] Gupta, M., Akiri, C., Aryal, K., Parker, E., and Praharaj, L. From chatgpt to threatgpt: Impact of generative ai in cybersecurity and privacy. *IEEE Access* (2023).

[22] Huh, M., Cheung, B., Wang, T., and Isola, P. The platonic representation hypothesis, 2024.

[23] Jain, N., Schwarzschild, A., Wen, Y., Somepalli, G., Kirchenbauer, J., yeh Chiang, P., Goldblum, M., Saha, A., Geiping, J., and Goldstein, T. Baseline defenses for adversarial attacks against aligned language models, 2023.

[24] Karl, F., and Scherp, A. Transformers are short text classifiers: A study of inductive short text classifiers on benchmarks and real-world datasets. *arXiv preprint arXiv:2211.16878* (2022).

[25] Kruse, R., Mostaghim, S., Borgelt, C., Braune, C., and Steinbrecher, M. Multi-layer perceptrons. In *Computational intelligence: a methodological introduction*. Springer, 2022, pp. 53–124.

[26] Kumar, D., AbuHashem, Y., and Durumeric, Z. Watch your language: Investigating content moderation with large language models, 2024.

[27] Lees, A., Tran, V. Q., Tay, Y., Sorensen, J., Gupta, J., Metzler, D., and Vasserman, L. A New Generation of Perspective API: Efficient Multilingual Character-level Transformers, Feb. 2022. arXiv:2202.11176 [cs].

[28] Lees, A., Tran, V. Q., Tay, Y., Sorensen, J., Gupta, J., Metzler, D., and Vasserman, L. A new generation of perspective api: Efficient multilingual character-level transformers, 2022.

[29] Li, J., Liu, Y., Liu, C., Ren, X., Shi, L., Sun, W., and Xue, Y. Self and cross-model distillation for llms: Effective methods for refusal pattern alignment, 2024.

[30] Li, J., Monroe, W., and Jurafsky, D. Understanding neural networks through representation erasure. *arXiv preprint arXiv:1612.08220* (2016).

[31] Li, Y., Liu, Y., Li, Y., Shi, L., Deng, G., Chen, S., and Wang, K. Lockpicking llms: A logit-based jailbreak using token-level manipulation, 2024.

[32] Liu, Y., Deng, G., Li, Y., Wang, K., Zhang, T., Liu, Y., Wang, H., Zheng, Y., and Liu, Y. Prompt injection attack against llm-integrated applications. *arXiv preprint arXiv:2306.05499* (2023).

[33] Liu, Y., Deng, G., Xu, Z., Li, Y., Zheng, Y., Zhang, Y., Zhao, L., Zhang, T., and Liu, Y. Jailbreaking chatgpt via prompt engineering: An empirical study. *arXiv preprint arXiv:2305.13860* (2023).

[34] Liu, Y., Yang, G., Deng, G., Chen, F., Chen, Y., Shi, L., Zhang, T., and Liu, Y. Groot: Adversarial testing for generative text-to-image models with tree-based semantic transformation, 2024.

[35] Markov, T., Zhang, C., Agarwal, S., Eloundou, T., Lee, T., Adler, S., Jiang, A., and Weng, L. A holistic approach to undesired content detection. *arXiv preprint arXiv:2208.03274* (2022).

[36] Mazeika, M., Phan, L., Yin, X., Zou, A., Wang, Z., Mu, N., Sakhaee, E., Li, N., Basart, S., Li, B., Forsyth, D., and Hendrycks, D. Harmbench: A standardized evaluation framework for automated red teaming and robust refusal, 2024.

[37] McInnes, L., Healy, J., and Melville, J. Umap: Uniform manifold approximation and projection for dimension reduction, 2020.

[38] McKnight, P. E., and Najab, J. Mann-whitney u test. *The Corsini encyclopedia of psychology* (2010), 1–1.

[39] Meta. "llama-13b". https://github.com/facebookresearch/llama/tree/llama_v1.

[40] Meta. "llama2-13b". https://github.com/facebookresearch/llama.

[41] Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research 21*, 140 (2020), 1–67.

[42] Si, W. M., Backes, M., Blackburn, J., De Cristofaro, E., Stringhini, G., Zannettou, S., and Zhang, Y. Why so toxic? measuring and triggering toxic behavior in open-domain chatbots. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security* (2022), pp. 2659–2673.

[43] Souly, A., Lu, Q., Bowen, D., Trinh, T., Hsieh, E., Pandey, S., Abbeel, P., Svegliato, J., Emmons, S., Watkins, O., and Toyer, S. A strongreject for empty jailbreaks, 2024.

[44] Team, G., Anil, R., Borgeaud, S., Wu, Y., Alayrac, J.-B., Yu, J., Soricut, R., Schalkwyk, J., Dai, A. M., Hauth, A., et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805* (2023).

[45] Team, G., Riviere, M., Pathak, S., Sessa, P. G., Hardin, C., Bhupatiraju, S., Hussenot, L., Mesnard, T., Shahriari, B., Ramé, A., Ferret, J., Liu, P., Tafti, P., Friesen, A., Casbon, M., Ramos, S., Kumar, R., Lan, C. L., Jerome, S., Tsitsulin, A., Vieillard, N., Stanczyk, P., Girgin, S., Momchev, N., Hoffman, M., Thakoor, S., Grill, J.-B., Neyshabur, B., Bachem, O., Walton, A., Severyn, A., Parrish, A., Ahmad, A., Hutchison, A., Abdagic, A., Carl, A., Shen, A., Brock, A., Coenen, A., Laforge, A., Paterson, A., Bastian, B., Piot, B., Wu, B., Royal, B., Chen, C., Kumar, C., Perry, C., Welty, C., Choquette-Choo, C. A., Sinopalnikov, D., Weinberger, D., Vijaykumar, D., Rogozińska, D., Herbison, D., Bandy, E., Wang, E., Noland, E., Moreira, E., Senter, E., Eltyshev, E., Visin, F., Rasskin, G., Wei, G., Cameron, G., Martins, G., Hashemi, H., Klimczak-Plucińska, H., Batra, H., Dhand, H., Nardini, I., Mein, J., Zhou, J., Svensson, J., Stanway, J., Chan, J., Zhou, J. P., Carrasqueira, J., Iljazi, J., Becker, J., Fernandez, J., van Amersfoort, J., Gordon, J., Lipschultz, J., Newlan, J., yeong Ji, J., Mohamed, K., Badola, K., Black, K., Millican, K., McDonell, K., Nguyen, K., Sodhia, K., Greene, K., Sjoesund, L. L., Usui, L., Sifre, L., Heuermann, L., Lago, L., McNealus, L., Soares, L. B., Kilpatrick, L., Dixon, L., Martins, L., Reid, M., Singh, M., Iverson, M., Görner, M., Velloso, M., Wirth, M., Davidow, M., Miller, M., Rahtz, M., Watson, M., Risdal, M., Kazemi, M., Moynihan, M., Zhang, M., Kahng, M., Park, M., Rahman, M., Khatwani, M., Dao, N., Bardoliwalla, N., Devanathan, N., Dumai, N., Chauhan, N., Wahltinez, O., Botarda, P., Barnes, P., Barham, P., Michel, P., Jin, P., Georgiev, P., Culliton, P., Kuppala, P., Comanescu, R., Merhej, R., Jana, R., Rokni, R. A., Agarwal, R., Mullins, R., Saadat, S., Carthy, S. M., Perrin, S., Arnold, S. M. R., Krause, S., Dai, S., Garg, S., Sheth, S., Ronstrom, S., Chan, S., Jordan, T., Yu, T., Eccles, T., Hennigan, T., Kocisky, T., Doshi, T., Jain, V., Yadav, V., Meshram, V., Dharmadhikari, V., Barkley, W., Wei, W., Ye, W., Han, W., Kwon, W., Xu, X., Shen, Z., Gong, Z., Wei, Z., Cotruta, V., Kirk, P., Rao, A., Giang, M., Peran, L., Warkentin, T., Collins, E., Barral, J., Ghahramani, Z., Hadsell, R., Sculley, D., Banks, J., Dragan, A., Petrov, S., Vinyals, O., Dean, J., Hassabis, D., Kavukcuoglu, K., Farabet, C., Buchatskaya, E., Borgeaud, S., Fiedel, N., Joulin, A., Kenealy, K., Dadashi, R., and Andreev, A. Gemma 2: Improving open language models at a practical size, 2024.

[46] Team, T. V. "vicuna-13b". https://github.com/lm-sys/FastChat.

[47] Wang, B., Chen, W., Pei, H., Xie, C., Kang, M., Zhang, C., Xu, C., Xiong, Z., Dutta, R., Schaeffer, R., Truong, S. T., Arora, S., Mazeika, M., Hendrycks, D., Lin, Z., Cheng, Y., Koyejo, S., Song, D., and Li, B. Decodingtrust: A comprehensive assessment of trustworthiness in gpt models, 2024.

[48] Xu, Z., Liu, Y., Deng, G., Li, Y., and Picek, S. A comprehensive study of jailbreak attack versus defense for large language models. In *Findings of the Association for Computational Linguistics ACL 2024* (Bangkok, Thailand and virtual meeting, Aug. 2024), L.-W. Ku, A. Martins, and V. Srikumar, Eds., Association for Computational Linguistics, pp. 7432–7449.

[49] Zeng, Y., Wu, Y., Zhang, X., Wang, H., and Wu, Q. Autodefense: Multi-agent llm defense against jailbreak attacks, 2024.

[50] Zhang, J., Wu, Q., Xu, Y., Cao, C., Du, Z., and Psounis, K. Efficient toxic content detection by bootstrapping and distilling large language models. In *Thirty-Eighth AAAI Conference on Artificial Intelligence, AAAI 2024, Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence, IAAI 2024, Fourteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2014, February 20-27, 2024, Vancouver, Canada* (2024), M. J. Wooldridge, J. G. Dy, and S. Natarajan, Eds., AAAI Press, pp. 21779–21787.

[51] Zhou, Y., Han, Y., Zhuang, H., Guo, T., Guo, K., Liang, Z., Bao, H., and Zhang, X. Defending jailbreak prompts via in-context adversarial game, 2024.

[52] Zhuo, T. Y., Huang, Y., Chen, C., and Xing, Z. Red teaming chatgpt via jailbreaking: Bias, robustness, reliability and toxicity. *arXiv preprint arXiv:2301.12867* (2023).

[53] Zou, A., Phan, L., Chen, S., Campbell, J., Guo, P., Ren, R., Pan, A., Yin, X., Mazeika, M., Dombrowski, A.-K., Goel, S., Li, N., Byun, M. J., Wang, Z., Mallen, A., Basart, S., Koyejo, S., Song, D., Fredrikson, M., Kolter, J. Z., and Hendrycks, D. Representation engineering: A top-down approach to ai transparency, 2023.

[54] Zou, A., Wang, Z., Kolter, J. Z., and Fredrikson, M. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043* (2023).